# Supplementary Materials to "*Assessing Parameter Identifiability in Agent-Based Models with Bayesian Amortized Inference*"

Francesco Renzini
University of Milan
JASSS, 29(4): 10

## 1 Hardware Specifications

We ran the estimation pipeline on a machine equipped with a 12-CPU 13th Gen Intel (R) Core (TM) i5-13420H (2.10 GHz) processor, without GPU acceleration. The Keras backend used was 'jax', and the system had 16GB of RAM available.

## 2 Learning the Posterior – Full Derivation

As described in Section 3 of the main manuscript, the network parameters $\phi$ are optimized during training by minimizing the Kullback–Leibler (KL) divergence between the true posterior and the approximate posterior, averaged over all possible datasets $\boldsymbol{x}$:

$$\hat{\phi} = \arg\min_{\phi} \mathbb{E}_{p(\boldsymbol{x})}[\mathbb{KL}(p(\theta \mid \boldsymbol{x}) \,\|\, p_\phi(\theta \mid \boldsymbol{x}))] \tag{1}$$

The KL divergence between two continuous probability densities $a(x)$ and $b(x)$ is defined as:

$$\mathbb{KL}(a \,\|\, b) = \int a(x)\, \log \frac{a(x)}{b(x)}\, dx = \mathbb{E}_{a(x)}[\log a(x) - \log b(x)] \tag{2}$$

Using Equation 2, we can expand Equation 1 and obtain:

$$\begin{aligned}
\hat{\phi} &= \arg\min_{\phi} \mathbb{E}_{p(\boldsymbol{x})}\left[\mathbb{E}_{p(\theta|\boldsymbol{x})}[\log\ p(\theta \mid \boldsymbol{x}) - \log\ p_\phi(\theta \mid \boldsymbol{x})]\right] && (3)\\
&= \arg\min_{\phi} \mathbb{E}_{p(\boldsymbol{x})}\left[\mathbb{E}_{p(\theta|\boldsymbol{x})}[-\log\ p_\phi(\theta \mid \boldsymbol{x})]\right] && (4)\\
&= \arg\min_{\phi} \int p(\boldsymbol{x})\, \left[\mathbb{E}_{p(\theta|\boldsymbol{x})}[-\log\ p_\phi(\theta \mid \boldsymbol{x})]\right] d\boldsymbol{x} && (5)\\
&= \arg\min_{\phi} - \int\int p(\boldsymbol{x}, \theta)\ \log\ p_\phi(\theta \mid \boldsymbol{x})\ d\boldsymbol{x}\ d\theta && (6)\\
&= \arg\min_{\phi} -\ \mathbb{E}_{p(\boldsymbol{x},\theta)}[\log\ p_\phi(\theta \mid \boldsymbol{x})] && (7)
\end{aligned}$$

In Equation 4, the term $\log p(\theta \mid \boldsymbol{x})$ can be omitted because the log-density of the true posterior does not depend on the neural network parameters $\phi$. Therefore, the optimization seeks parameters $\phi$ that minimize the expected negative log-density assigned by the

approximation $p_\phi(\theta \mid \boldsymbol{x})$ to the true parameter values $\theta$, conditional on the observed data $\boldsymbol{x}$. Equivalently, we seek a conditional density $p_\phi(\theta \mid \boldsymbol{x})$ that best approximates the true posterior over the prior predictive distribution (Equation 7), where each pair $(\boldsymbol{x}, \theta)$ is generated by first sampling $\theta$ from the prior (i.e., $\theta \sim p(\theta)$) and then simulating $\boldsymbol{x}$ through the generative mechanism $g(\cdot)$ (Radev et al., 2020; Min et al., 2026).

Equation 6 can also be formulated as a maximization problem:

$$\hat{\phi} = \arg\max_{\phi} \int \int p(\boldsymbol{x}, \theta)\ \log p_\phi(\theta \mid \boldsymbol{x})\, d\boldsymbol{x}\, d\theta \tag{8}$$

which yields the same optimal parameters $\hat{\phi}$ under standard optimization algorithms.

By construction of the normalizing flow, $\boldsymbol{z}$ is the output of the forward transformation $f_\phi(\theta, \boldsymbol{x})$, while $\theta$ is obtained by applying the inverse transformation $f_\phi^{-1}(\boldsymbol{z}, \boldsymbol{x})$. Therefore, by the change-of-variables formula, $p_\phi(\theta \mid \boldsymbol{x})$ can equivalently be written as:

$$p_\phi(\theta \mid \boldsymbol{x}) = p(\boldsymbol{z} = f_\phi(\theta, \boldsymbol{x})) \left| \det \left( \frac{\partial f_\phi(\theta, \boldsymbol{x})}{\partial \theta} \right) \right| \tag{9}$$

where $\frac{\partial f_\phi(\theta, \boldsymbol{x})}{\partial \theta}$ denotes the Jacobian matrix of $f_\phi(\cdot)$ with respect to $\theta$, evaluated at $(\theta, \boldsymbol{x})$, and can be more concisely denoted by $J_{f_\phi}$.

Substituting the expression for the approximate posterior given in Equation 9 into Equation 6, we obtain:

$$\hat{\phi} = \arg\min_{\phi} - \int \int p(\boldsymbol{x}, \theta)\ (\log\ p(\boldsymbol{z} = f_\phi(\theta, \boldsymbol{x})) + \log\ \left| det J_{f_\phi} \right|)\ dx\ d\theta \tag{10}$$

Using simulations from the prior predictive distribution, we construct batches of size $M$ consisting of simulated datasets and their corresponding generating behavioral parameters, $\{(\boldsymbol{x}^{(m)}, \theta^{(m)})\}_{m=1}^{M}$, and optimize a Monte Carlo approximation of Equation 10:

$$\hat{\phi} = \arg\min_{\phi} \frac{1}{M} \sum_{m=1}^{M} \left( -\log p\Big(\boldsymbol{z}^{(m)} = f_\phi(\theta^{(m)}, \boldsymbol{x}^{(m)})\Big) - \log \left| \det J_{f_\phi}^{(m)} \right| \right) \tag{11}$$

$$= \arg\min_{\phi} \frac{1}{M} \sum_{m=1}^{M} \left( \frac{\left\| f_\phi(\theta^{(m)}, \boldsymbol{x}^{(m)}) \right\|_2^2}{2} - \log \left| \det J_{f_\phi}^{(m)} \right| \right) \tag{12}$$

where the second equality (Equation 12) arises from the assumption that the latent variable $\boldsymbol{z}$ follows a standard multivariate Gaussian distribution with zero mean and unit covariance. Under this assumption, the negative log-density of $\boldsymbol{z}$ is proportional to $\|\boldsymbol{z}\|_2^2$, up to an additive constant, which can be safely ignored during optimization. By design, $\boldsymbol{z} = f_\phi(\theta, \boldsymbol{x})$, yielding the quadratic term in Equation 12. The second term in Equation 12 corresponds to the log-determinant of the Jacobian of the transformation and controls the rate of volume change induced by the nonlinear, invertible mapping from $\theta$ to $\boldsymbol{z}$ achieved by $f_\phi$.

By substituting $\boldsymbol{x}^{(m)}$ in Equation 12 with the learned summary representation $h_\psi(\boldsymbol{x}^{(m)})$ generated by the summary network, and optimizing jointly with respect to the cINN parameters $\phi$ and the summary network parameters $\psi$, we obtain Equation 8 of the main manuscript.

## 3 Simulation Based Calibration (SBC) – Technical Derivation

The theoretical foundation of the uniform rank statistics test in SBC lies in the self-consistency of the Bayesian joint distribution (Talts et al., 2018). We begin by recalling the simulation procedure used to compute the rank statistic.

We simulate $N$ datasets $\{\boldsymbol{x}^{(i)}\}_{i=1}^N$ by drawing $N$ parameter vectors $\{\theta^{(i)}\}_{i=1}^N$ from the prior distribution and passing each of them through the generative mechanism $g(\cdot)$. The sampled parameter vectors are retained as the corresponding "ground truth" values. Each simulated dataset is then treated as an observed dataset $\boldsymbol{x}^*$. For every $\boldsymbol{x}^{(i)}$, we draw $L$ latent vectors $\{\boldsymbol{z}_\ell\}_{\ell=1}^L$ from the multivariate standard Gaussian and pass them, together with $\boldsymbol{x}^{(i)}$, through the inverse transformation of the trained normalizing flow to obtain samples from the approximate posterior distribution $p_{\hat{\phi},\hat{\psi}}(\theta \mid \boldsymbol{x}^{(i)})$.

We then compute the rank statistic $r^{(i)}$ by ranking the ground truth parameter $\theta^{(i)}$ among the $L$ posterior samples. Specifically, $r^{(i)}$ is defined as the number of posterior samples that are less than or equal to $\theta^{(i)}$ (Talts et al., 2018; Betancourt, 2020). If the inference procedure is well calibrated, the resulting rank statistics $\{r^{(i)}\}_{i=1}^N$ are expected to follow an approximately discrete uniform distribution over $\{0, 1, \ldots, L\}$ (Talts et al., 2018).

We now derive why uniformly distributed rank statistics provide evidence that the self-consistency property of the Bayesian joint distribution is satisfied and, consequently, that the posterior uncertainty and coverage obtained through `BayesFlow` are well calibrated. By definition:

$$p(\theta, \boldsymbol{x}) = p(\boldsymbol{x} \mid \theta)\, p(\theta) = p(\theta \mid \boldsymbol{x})\, p(\boldsymbol{x}) \tag{13}$$

Marginalizing the joint distribution over all possible datasets recovers the prior distribution:

$$p(\theta) = \int p(\theta, \boldsymbol{x})\, dx = \int p(\boldsymbol{x} \mid \theta)\, p(\theta)\, dx = p(\theta) \underbrace{\int p(\boldsymbol{x} \mid \theta)\, dx}_{=1} \tag{14}$$

Therefore, by the identities of Equation 13, we obtain:

$$p(\theta) = \int p(\theta \mid \boldsymbol{x})\, p(\boldsymbol{x})\, dx \tag{15}$$

Equation 15 implies that the prior distribution can be recovered by averaging the posterior distribution over datasets sampled from the prior predictive distribution. Consequently, when the posterior approximation is calibrated, a parameter value $\theta^{(i)}$ sampled from the prior and used to generate the corresponding dataset $\boldsymbol{x}^{(i)}$ behaves as a 'fair draw' from the

corresponding posterior distribution conditioned on $\boldsymbol{x}^{(i)}$. Consequently, for each of the $N$ simulated datasets, we record the rank of $\theta^{(i)}$ among samples drawn from the inferred posterior. If Equation 15 holds, the resulting distribution of rank statistics is discrete uniform over $\{0, 1, \ldots, L\}$ (Talts et al., 2018; Radev et al., 2021).

# 4 Pathological Distributions of Normalized Ranks and Their ECDF Difference Plots - SBC

Figure S1 (top row) shows prototypical pathological distributions of the normalized rank statistics used in Simulation-Based Calibration (SBC) as histograms, while the bottom row displays the corresponding ECDF difference plots. Both the distributions and the ECDF differences indicate the presence of errors in the estimation phase.

Note that these distributions were generated to illustrate specific pathological shapes and were not derived from an actual model. Consequently, no confidence bands are overlaid on the ECDF plots.

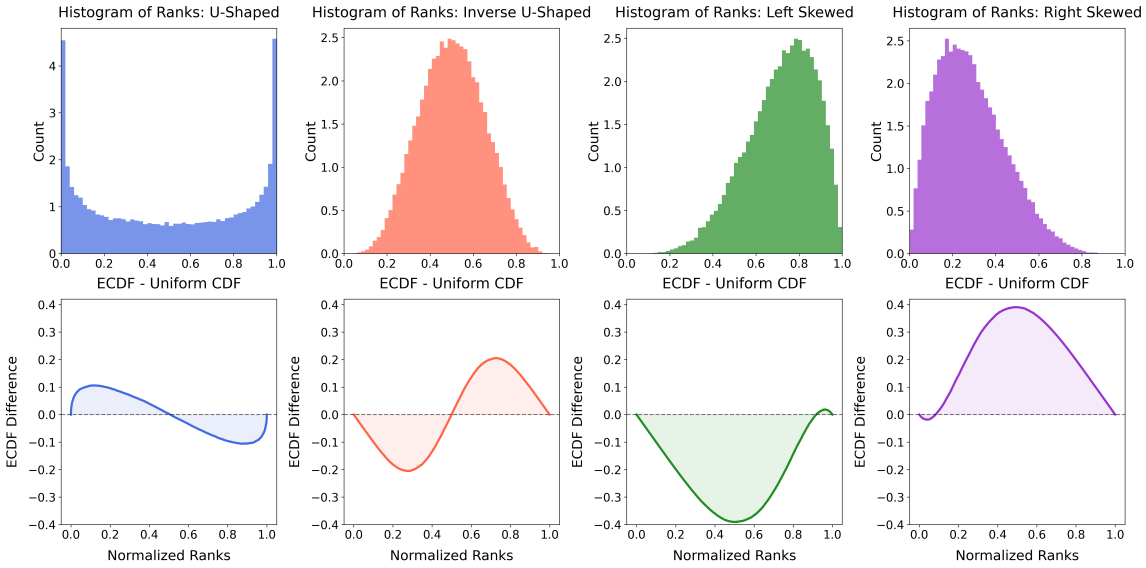



Figure S1: Top row: Histograms of prototypical pathological distributions of the normalized rank statistics used in SBC. Bottom row: Corresponding ECDF difference plots. For simplicity, no confidence bands are overlaid on the ECDF differences.

A U-shaped distribution of ranks indicates an overconfident posterior that is too narrow (*underdispersed*) relative to the true distribution (Talts et al., 2018; Betancourt, 2020; Säilynoja et al., 2021), producing an ECDF plot with an excess of low ranks and a deficit of

high ranks compared to the theoretical uniform distribution (Modrák, 2025). In contrast, an inverse U-shaped (bell-shaped) distribution of ranks suggests an underconfident, overly conservative posterior (Talts et al., 2018; Säilynoja et al., 2021), resulting in an ECDF with too few low ranks and too many high ranks (Modrák, 2025). Asymmetric rank distributions signal a systematic bias in the posterior estimates (Talts et al., 2018; Säilynoja et al., 2021), which manifests as an asymmetric, inverse U-shaped ECDF in case of overestimation, while underestimation produces an asymmetric, U-shaped ECDF (Modrák, 2025).

# 5 Details on the DeepSet Architecture Used for the Bounded Confidence Model

As outlined in the main manuscript, the DeepSet architecture learns embeddings of opinion distributions through a two-stage procedure. In the first stage, we apply a stack of two modules to the set of opinions $\{\hat{o_i}^T\}_{i=1}^{200}$, generated from a given value of $\epsilon_{max}$. Within the first module, each scalar opinion $\hat{o_i}^T \in [0, 1]$ is initially projected into a 64-dimensional latent space to form a latent vector $h_i^{(1)} \in \mathbb{R}^{64}$. This higher-dimensional representation provides a richer feature space in which the network can capture complex relationships within an opinion distribution. It is then refined by a multilayer perceptron (MLP) consisting of two 64-unit hidden layers implemented as residual blocks to ensure numerical stability and efficient gradient flow. The resulting 64-dimensional output is subsequently processed by a standalone $64 \times 64$ dense layer. We denote the final processed latent representation for each agent by $\hat{h}_i^{(1)} \in \mathbb{R}^{64}$.

To encode information about the overall distribution, we then compute a global context vector $\bar{c}^{(1)}$. This is achieved by passing the processed latent vectors $\hat{h}_i^{(1)}$ through a bottleneck MLP (consisting of a 64-unit residual layer and a final 4-unit dense projection) and applying mean pooling to the resulting compressed representations:

$$\begin{gathered} \bar{c}^{(1)} = \frac{1}{200} \sum_{i=1}^{200} \mathrm{MLP}_{\mathrm{bottleneck}}(\hat{h}_i^{(1)}) \\ \text{where: } \hat{h}_i^{(1)} \in \mathbb{R}^{64} \text{ , } \mathrm{MLP}_{\mathrm{bottleneck}}(\hat{h}_i^{(1)}) \in \mathbb{R}^4 \end{gathered} \tag{16}$$

Because mean pooling is commutative, the context vector $\bar{c}^{(1)}$ is invariant to the ordering of opinions. This context vector is then concatenated with each agent's individual latent vector $\hat{h}_i^{(1)}$, yielding an augmented 68-dimensional representation. The augmented vector is subsequently processed by a secondary MLP, consisting of two 64-unit hidden layers with residual blocks, followed by an additional $64 \times 64$ dense layer, to produce updated representations $\hat{h}_{i,\mathrm{updated}}^{(1)} \in \mathbb{R}^{64}$. By fusing $\hat{h}_i^{(1)}$ with the 4-dimensional global summary, each updated representation jointly encodes both the agent's own opinion and salient information about the overall opinion distribution.

The second module takes the updated vectors $\hat{h}^{(1)}_{i,\text{updated}}$ as input and applies the same sequence of operations. This module enables each agent's representation to capture more complex and higher-order features of the opinion distribution. Consequently, each final latent vector $\hat{h}^{(2)}_{i,\text{updated}}$ encodes rich information about the overall opinion distribution while preserving the agent-wise structure: permuting the input opinions in the first module doesn't change the final latent representations $\hat{h}^{(2)}_{i,\text{updated}}$.

In the second stage, a *permutation-invariant* module aggregates the outputs of the first stage, $\{\hat{h}^{(2)}_{i,\text{updated}}\}_{i=1}^{200}$. Each of the 200 latent vectors is processed by two 64-unit MLPs with residual blocks, followed by $64 \times 64$ dense layers, before being aggregated into a single 64-dimensional global representation via mean pooling. This global representation is then mapped to the final vector $\tilde{\boldsymbol{x}} \in \mathbb{R}^M$, with $M < 64$, through a linear output projection. In this way, the original population-level opinion distribution is compressed into an $M$-dimensional embedding of real numbers. In the main manuscript we set $M = 16$; below we also present results for $M = 24$.

All learnable parameters of the MLPs and dense layers in the two-stage architecture, together with those of the final linear projection that maps the 64-dimensional representation to the $M$-dimensional summary vector, as well as the parameters of the layer normalizations and internal data-processing components between modules, collectively define the parameter set $\psi$ of the summary network. In total, the network comprises 89856 learnable parameters when $M = 16$. The Sigmoid Linear Unit (SiLU) activation function is used throughout the network, and a dropout rate of 0.05 is applied for regularization within the MLP layers (BayesFlow, 2026).

The number of modules in the first stage (2 in our implementation), the architectures of the MLPs and dense layers, the dimensionality of the latent representations (64), the choice of pooling operation, and the bottleneck size (4) are all hyperparameters that can be tuned. In this work, we adopt the default values provided by BayesFlow (2026).

# 6 Estimating the Modified Bounded Confidence Model with a Different $\tilde{\boldsymbol{x}}$

In this section, we present results obtained using the same DeepSet architecture as in the main manuscript to learn a lower-dimensional embedding of the perturbed opinion distributions. Here, however, the summary representation is expanded to 24 real-valued numbers instead of 16. In other words, the pipeline that transforms the opinion distribution into a lower-dimensional vector encoding its key characteristics now produces a final representation vector $\tilde{\boldsymbol{x}}$ of dimensionality 24 rather than 16.

Figures S2 and S3 yield results identical to Figures 2 and 3 in the main manuscript. The posterior of $\epsilon_{max}$ remains well-calibrated, and the three characteristic regimes are still clearly discernible.

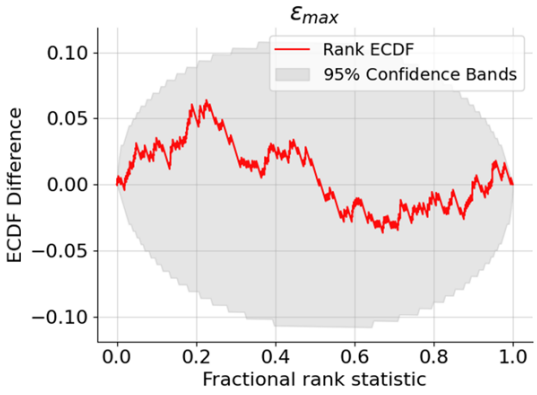



Figure S2: Simulation-based calibration (SBC results for the posterior distribution of $\epsilon_{max}$, when $\tilde{\boldsymbol{x}}$ is a 24-dimensional vector. The red curve shows the difference between the normalized empirical cumulative distribution function (ECDF) of the rank statistics of the ground truth $\epsilon_{max}$ values within their corresponding posterior samples and the cumulative distribution function of a normalized discrete uniform distribution over ranks. The grey bands represent 95% confidence intervals under finite-sample variability. Deviations fluctuating around zero indicate that the inferred posteriors are well calibrated.

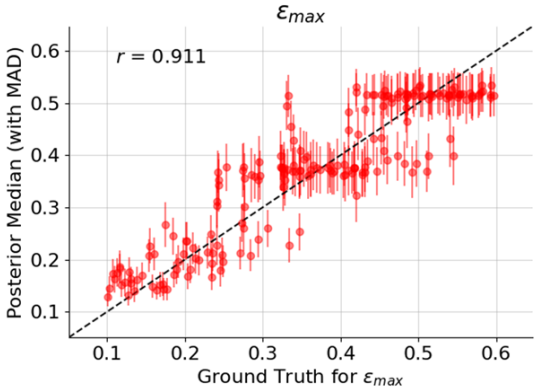



Figure S3: Each point shows a ground truth value of $\epsilon_{max}$ plotted against the posterior median inferred from synthetic data generated using that same ground truth value, when $\tilde{\boldsymbol{x}}$ is a 24-dimensional vector. Error bars represent the posterior median absolute deviation (MAD). The dotted diagonal line corresponds to perfect recovery. $r$ denotes the Pearson correlation coefficient between ground truth values and posterior median estimates.

# 7 Details on the Summary Network Architectures Used for the SIR Model

We begin by describing the GRU architecture (Chung et al., 2014; Radev, 2026) used to learn a lower-dimensional embedding of the time series of noisy new infected cases. A hidden state vector $\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^{64}$ is initialized and updated sequentially as the architecture processes the time series $\boldsymbol{x} = (x_{t_1}, x_{t_2}, \ldots, x_{t_{50}})$ (Radev, 2026). At each time step $t \in \{1, \ldots, 50\}$, the hidden state is updated via three learned gating mechanisms:

- *Update gate* ($\mathbf{y}_t$): controls how much new information is incorporated into the hidden state;
- *Reset gate* ($\mathbf{r}_t$): determines how much past information is discarded when computing the candidate hidden state;
- *Candidate state* ($\tilde{\mathbf{h}}_t$): proposes updated content based on the current input and the selectively reset past state.

Together, these mechanisms enable the network to learn temporal patterns relevant for inference — for instance, capturing long-range dependencies that link early epidemic dynamics (e.g., around time step 10) to later behavior (e.g., around time step 40), or identifying characteristic features such as peaks, troughs, and epidemic waves.

Mathematically, at each time step $t$, the gates are computed as:

$$\begin{aligned} \mathbf{y}_t &= \sigma(\mathbf{w}_y x_t + \mathbf{U}_y \mathbf{h}_{t-1} + \mathbf{b}_y) \\ \mathbf{r}_t &= \sigma(\mathbf{w}_r x_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{w}_h x_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \end{aligned} \tag{17}$$

$\mathbf{w}_y, \mathbf{w}_r, \mathbf{w}_h \in \mathbb{R}^{64}$ are learnable weight vectors,[1] $\mathbf{U}_y, \mathbf{U}_r, \mathbf{U}_h \in \mathbb{R}^{64\times 64}$ are learnable recurrent weight matrices, and $\mathbf{b}_y, \mathbf{b}_r, \mathbf{b}_h \in \mathbb{R}^{64}$ are learnable bias vectors. Here, $\sigma(\cdot)$ denotes the sigmoid activation function, $\tanh(\cdot)$ the hyperbolic tangent, and $\odot$ element-wise multiplication. The hidden state is then updated as:

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{y}_t) \odot \mathbf{h}_{t-1} + \mathbf{y}_t \odot \tilde{\mathbf{h}}_t \tag{18}$$

where $\mathbf{1} \in \mathbb{R}^{64}$ is a vector of ones. This update rule implements a learnable interpolation between the previous hidden state ($\mathbf{h}_{t-1}$) and the new candidate state ($\tilde{\mathbf{h}}_t$), with the update gate $\mathbf{y}_t$ controlling the balance.

Specifically, when the $i_{\text{th}}$ element of the update gate is $y_{t,i} \approx 1$, the corresponding element in the new hidden state $h_{t,i}$ is primarily determined by the new candidate $\tilde{h}_{t,i}$

[1] Or, equivalently, $64 \times 1$ matrices. This second notation makes dimensional bookkeeping more transparent.

(effectively "updating" with new information). Conversely, when $y_{t,i} \approx 0$, the element $h_{t,i}$ retains the previous value $h_{t-1,i}$ (effectively "remembering" past information). This selective update mechanism allows the GRU to adaptively decide which dimensions of the hidden state to update based on the current input and context.

After processing all 50 time steps, the final state $\mathbf{h}_{50} \in \mathbb{R}^{64}$ encodes the complete temporal structure of the epidemic curve. This representation is then linearly compressed to an 8-dimensional summary vector (Lueckmann et al., 2017; Radev, 2026):

$$\tilde{\boldsymbol{x}} = \mathbf{W}_{\text{dense}}^{\top}\mathbf{h}_{50} + \mathbf{b}_{\text{dense}} \in \mathbb{R}^{8} \tag{19}$$

where $\mathbf{W}_{\text{dense}} \in \mathbb{R}^{64\times 8}$ and $\mathbf{b}_{\text{dense}} \in \mathbb{R}^{8}$ are additional learnable parameters.

This compressed representation $\tilde{\boldsymbol{x}}$ encapsulates relevant temporal features, including peaks, troughs, growth rates, and wave patterns, in a compact 8-dimensional vector.

The GRU architecture comprises a total of 13384 trainable parameters. During training, a dropout rate of 10% is applied to the hidden-state activations as a regularization strategy. The hidden-state dimensionality (64) and the dimensionality of the learned summary representation $\tilde{\boldsymbol{x}}$ (8) are tunable hyperparameters. In this work, we adopt the default values provided by BayesFlow (2026).

To process the multivariate time series of noisy infected and recovered cases, following Zhang and Mikelsons (2023) and BayesFlow (2026), we employ an architecture that combines a convolutional neural network (CNN) for local feature extraction with a GRU-based summary network, followed by a linear projection layer.

The input $\boldsymbol{x}$ is now represented as a $2\times 50$ matrix. A convolutional layer first operates on this matrix, extracting local features from the joint temporal dynamics of the two incidence series. The extracted features are then passed to a GRU with a hidden state vector $\mathbf{h}_t$ of dimension 128, larger than in the univariate case to provide greater representational capacity for modeling multivariate temporal dependencies. The convolutional layer employs 32 filters, a kernel size of 3, a stride of 1, and the Mish activation function; a dropout rate of 5% is applied to the GRU hidden states for regularisation (BayesFlow, 2026). The final linear projection maps the GRU output to a 16-dimensional vector $\tilde{\boldsymbol{x}}$ (BayesFlow, 2026).

Overall, the resulting summary network contains a total of 132976 trainable parameters. The increase relative to the univariate summary network is driven by the inclusion of the CNN, the additional preprocessing and projection layers between the CNN and the GRU, and the higher dimensionality of both the GRU's internal hidden state and the learned summary representation $\tilde{\boldsymbol{x}}$.

## 8 Impact of the Reporting Delay on the SIR Model

To illustrate the differential impact of the reporting delay $D$ on the time series of newly infected cases, we proceeded as follows. We fixed $\lambda = 0.50$, $\mu = 0.10$, and initialized the simulation with 150 "patient zero" agents. We then generated three scenarios corresponding

to $D = \{5, 10, 15\}$. For each scenario, we simulated 500 time series of newly infected cases (Figure S4).

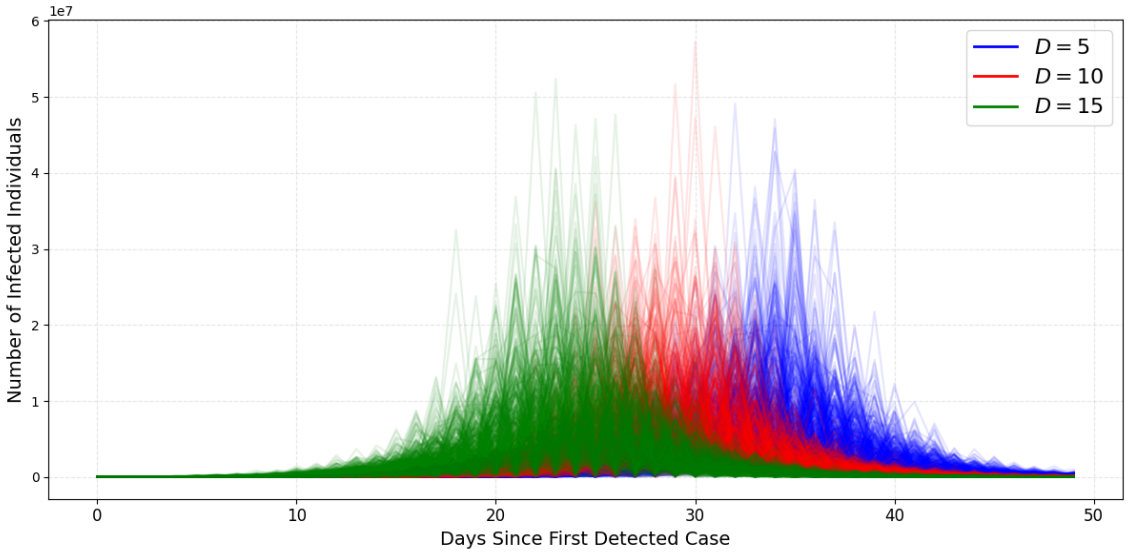



Figure S4: 500 time series of daily new infected cases for different values of $D$.

We observe that increasing $D$ causes the observed peak of infections to occur earlier. If reporting of cases is delayed by two weeks while the underlying epidemic dynamics continue to unfold, the observed peak will appear earlier relative to the first report of a case compared to a scenario with $D = 5$. Importantly, the underlying epidemic dynamics remain identical across scenarios. The difference lies solely in the observation window: for $D = 15$, infections are recorded from day 15 to day 65; for $D = 10$, from day 10 to day 60; and for $D = 5$, from day 5 to day 55.

## 9 Comparing Predictions from `BayesFlow` and a Differential Evolution Optimizer

In some applications, modelers may primarily be interested in the ability of an estimation algorithm to reproduce a given dataset in-sample and, where possible, to generalize out-of-sample. As shown in Figures 6 and 15 of the main manuscript, `BayesFlow`-based posterior inference retains high in-sample predictive performance. Figure S5 further shows that the inferred parameters can generate accurate out-of-sample predictions. Specifically, we estimated the parameters of the SIR model using only the time series of infected cases, which

were compressed into lower-dimensional embeddings through a GRU, for four evaluation datasets (up to $T = 50$). We then used the inferred parameters to predict the subsequent 50 time steps of infected cases and found a close agreement between the predicted trajectories and the previously unseen simulated observations. We recall that, under this model specification, both $\mu$ and $D$ remain non-identifiable (see the main manuscript), highlighting that good predictive performance does not necessarily imply parameter identifiability.

However, when predictive accuracy is the primary objective, a full Bayesian amortized inference framework may not always be necessary. A wide range of metaheuristic and optimization-based approaches can identify single parameter combinations that achieve good in-sample fit and, in some settings, comparable out-of-sample predictive performance. For instance, Figure S5 compares the predictive performance of `BayesFlow` with that of a differential evolution optimizer (Storn and Price, 1997)[2] on the same evaluation datasets used for the SIR model. For this specific predictive task, both approaches achieve comparable predictive performance.

[2]We used the `sciPy` implementation. See: `https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html`

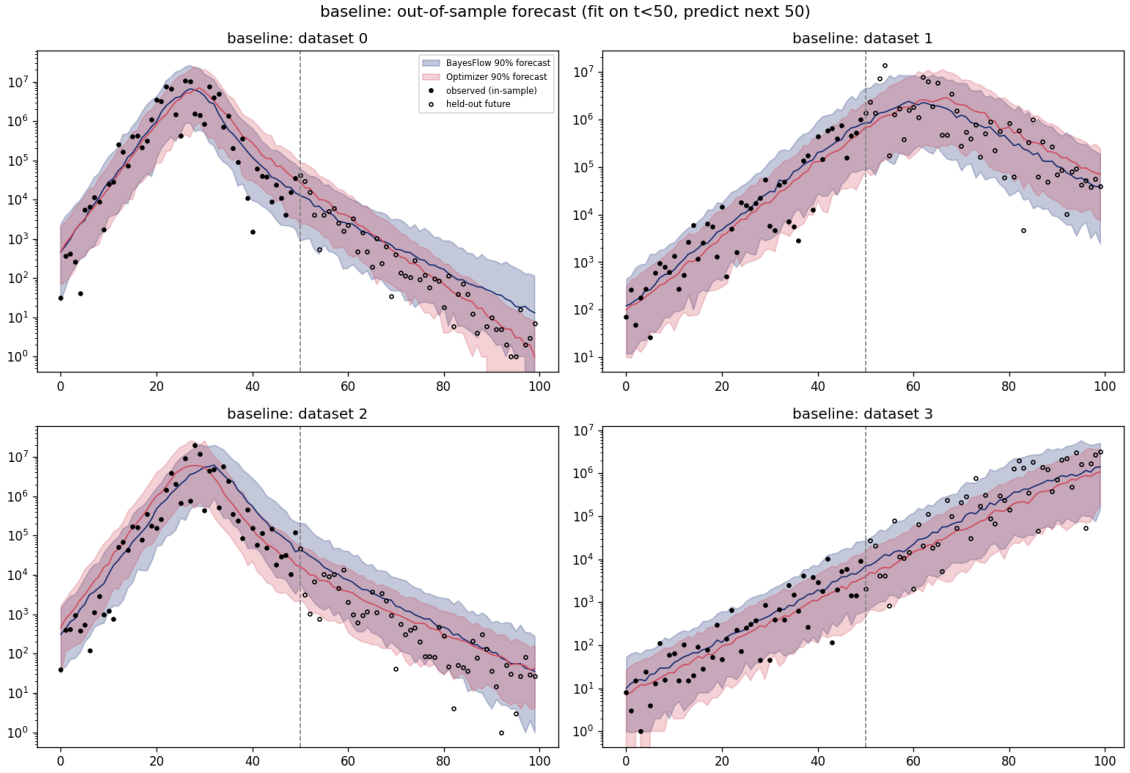



Figure S5: In-sample fit and out-of-sample predictive performance of `BayesFlow` and a differential evolution optimizer across four evaluation datasets. The number of infected cases over time is shown on a logarithmic scale.

## References

BayesFlow (2026). Documentation for BayesFlow network architectures. Available at: `https://bayesflow.org/v2.0.12/api/bayesflow.networks.html`.

Betancourt, M. (2020). Towards a principled Bayesian workflow. Available at: `https://betanalpha.github.io/assets/case_studies/principled_bayesian_workflow.html`.

Chung, J., Gulcehre, C., Cho, K. H., and Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint. Available at: `https://arxiv.org/pdf/1412.3555`.

Lueckmann, J. M., Goncalves, P. J., Bassetto, G., Öcal, K., Nonnenmacher, M., and Macke,

J. H. (2017). Flexible statistical inference for mechanistic models of neural dynamics. arXiv preprint. Available at: https://arxiv.org/abs/1711.01861.

Min, J., Ning, Y., Pope, N. S., Baumdicker, F., and Kern, A. D. (2026). Neural posterior estimation for population genetics. *Genetics*, 233(3):iyag107.

Modrák, M. (2025). SBC rank visualizations. Available at: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html.

Radev, S. T. (2026). Posterior estimation for SIR-like models. BayesFlow documentation. Available at: https://bayesflow.org/v2.0.12/_examples/SIR_Posterior_Estimation.html.

Radev, S. T., Graw, F., Chen, S., Mutters, N. T., Eichel, V. M., Bärnighausen, T., and Köthe, U. (2021). OutbreakFlow: Model-based Bayesian inference of disease outbreak dynamics with invertible neural networks and its application to the COVID-19 pandemics in Germany. *PLoS Computational Biology*, 17(10):e1009472.

Radev, S. T., Mertens, U. K., Voss, A., Ardizzone, L., and Köthe, U. (2020). BayesFlow: Learning complex stochastic models with invertible neural networks. arXiv preprint. Available at: https://arxiv.org/abs/2003.06281.

Säilynoja, T., Bürkner, P. C., and Vehtari, A. (2021). Graphical test for discrete uniformity and its applications in goodness of fit evaluation and multiple sample comparison. arXiv preprint. Available at: https://arxiv.org/abs/2103.10522.

Storn, R. and Price, K. (1997). Differential Evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11:341–359.

Talts, S., Betancourt, M., Simpson, D., Vehtari, A., and Gelman, A. (2018). Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint. Available at: https://arxiv.org/abs/1804.06788.

Zhang, Y. and Mikelsons, L. (2023). Solving stochastic inverse problems with stochastic BayesFlow. IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), Seattle, WA, USA.